

SPI NAND FLASH

使用说明

1 切换 SPI NAND FLASH 配置

系统默认配置为 SPI NOR FLASH，切换为 SPI NAND FLASH 需要进行以下操作：
打开 AMTLDR、STEPLDR 两个工程中的 amt630h.h 和 amt630hv100-freertos 工程中的 board.h，修改配置 DEVICE_TYPE_SELECT 为 SPI_NAND_FLASH，并根据自身需求调整各分区大小。

```
/****** flash type configuration *****/
#define SPI_NOR_FLASH      0
#define SPI_NAND_FLASH    1
#define EMMC_FLASH        2
#define DEVICE_TYPE_SELECT SPI_NAND_FLASH
/******

/****** update address configuration *****/
#if DEVICE_TYPE_SELECT == SPI_NAND_FLASH
#define LOADER_OFFSET      0x0
#define STEPLDRA_OFFSET   0x20000
#define STEPLDRB_OFFSET   0x40000
#define STEPLDR_MAX_SIZE   0x14000
#define SYSINFOA_MEDIA_OFFSET 0x60000
#define SYSINFOB_MEDIA_OFFSET 0x80000
#define UPDATEFILE_MEDIA_OFFSET 0xa0000
#define UPDATEFILE_MEDIA_B_OFFSET 0xc00000
#define UPDATEFILE_MAX_SIZE 0xb00000
#define OTA_MEDIA_OFFSET   0x1800000
#define OTA_MEDIA_SIZE     0x800000
#define SPI0_QSPI_MODE
```

2 SPI NAND 缓存机制说明

SPI NOR 和 SPI NAND 在数据写入前都需要擦除操作，SPI NAND 擦除操作的最小单位为块（默认 128KB），SPI NOR 擦除操作的最小单位为扇区（默认 4KB）。为兼容两者的操作接口（即文件系统或应用程序调用接口时不需要区分 NOR 和 NAND），并提高 SPI NAND 的读写效率，amt630hv100-freertos 工程的驱动中引入了缓存机制（AMTLDR 和 STEPLDR 工程不需要），当系统调用擦/写函数时，驱动不会实时进行擦/写 Flash 操作，而是先缓存要写的数据，然后把任务推送到定时刷新线程中，缓存数据超过 30 秒后才会被刷新（写入 Flash）。

引入缓存机制后，写数据时不会实时写到 Flash 中，当系统重启或者异常断电时，如果写操作后的时间不到 30 秒，数据就不会同步写入 Flash，因此会造成数据丢失。所以应用程序在写操作结束后，需要调用 sfud_flush() 接口来实时刷新数据到 Flash。

① 该工具不同版本下有差异，运行不同版本的工程应用时需要用该版本下的工具重新生成 rom.bin 并升级。

sfud_flush()同步接口的使用场景列举如下：

场景 1：OTA/UART 升级

OTA 升级或串口升级结束后，应用程序需要调用 sfud_flush()刷新缓存，避免异常断电或系统重启时，数据未完全同步写入 Flash，导致升级失败。由于升级成功后会保存 sysinfo 信息，所以在 SaveSysInfo()函数中调用 sfud_flush()即可：

```
void SaveSysInfo(void)
{
    amt630h_sysinfo.checksum = xcrc32((unsigned char*)&amt630h_sysinfo, sizeof(SysInfo));
    #if DEVICE_TYPE_SELECT != EMMC_FLASH
        sfud_flash *sflash = sfud_get_device(0);
        if(sflash) {
            sfud_erase_write(sflash, SYSINFOB_MEDIA_OFFSET, sizeof(SysInfo), (void*)&amt630h_sysinfo);
            sfud_erase_write(sflash, SYSINFOA_MEDIA_OFFSET, sizeof(SysInfo), (void*)&amt630h_sysinfo);
            #if DEVICE_TYPE_SELECT == SPI_NAND_FLASH
                sfud_flush(sflash);
            #endif
        }
    #else
        emmc_write(SYSINFOB_MEDIA_OFFSET, sizeof(SysInfo), (void*)&amt630h_sysinfo);
        emmc_write(SYSINFOA_MEDIA_OFFSET, sizeof(SysInfo), (void*)&amt630h_sysinfo);
    #endif
}
```

如果在保存 sysinfo 信息前需要重新读出来升级数据进行校验，跟写入数据做对比来判断是否升级成功，那么数据写入后（在读取前）需要调用 sfud_flush()进行同步缓存。

场景 2：通过文件系统接口保存文件

应用程序通过文件系统接口修改或保存文件后，需要调用 sfud_flush()刷新缓存。主要分为文件和目录操作两类接口：

(1)、文件写操作

在文件写（ff_write）操作完成，文件关闭（ff_close）之后，需要调用 sfud_flush()刷新缓存。只进行文件读（ff_read）操作之后不需要刷新缓存。

(2)、目录操作

ff_rmdir/ff_mkdir/ff_remove/ff_rename 等操作后，如果没有文件写操作，需要调用 sfud_flush()刷新缓存。

场景 3：通过 Flash 偏移地址直接保存数据

应用程序直接从指定的偏移地址开始保存数据（即直接调用 sfud_erase_write()或者 sfud_write()接口在 Flash 的固定偏移地址处保存数据）时，数据保存完成后，需要调用 sfud_flush()刷新缓存。另外，此种数据保存方式下，应用程序需要单独划分一段 Flash 区域，避免同其他分区产生地址冲突，并且此区域的起始地址必须是块对齐的。

3 SPI NAND 坏块表和坏块替换块

SPI NAND 坏块表存放在 Flash 的最后 4 个块中，其中前两个块分别保存一份坏块表，互为备份，后两个块作为坏块表的坏块替换块。

Flash 的数据存储区域采用替换块的方式来处理坏块，紧临坏块表区域预留了 2% 的容量作为坏块替换块，当发现坏块时，系统将从此区域查找可用块来替代坏块，并在坏块表中建立坏块和替换块的映射关系。

Flash 可用容量 = 总容量 - 坏块表容量(4 blocks) - 替换块容量 (2%)

可用容量	替换块 (2%)	坏块表 (4)
------	----------	---------

系统分区的时候，注意使用可用容量，而不是总容量。

此外，如果 Flash 容量充足，需要增大备份块数量的话，注意需要对 AMTLDR、STEPLDR、amt630hv100-freertos 三个工程同时进行修改，修改如下的宏定义：

```
#define SNFUD_REPLACE_BLK_PERCENT 2 //default: 2%
```

4 SPI NAND 分区要求

SPI NAND 各分区的起始地址和大小必须是块对齐的。源码中 SPI NAND FLASH 各分区的起始地址和大小默认是块对齐的，后续如果需要修改分区或者添加新分区，分区的起始地址和大小必须是块对齐的。

系统分区时使用 Flash 的可用容量，而不是总容量。SPI NAND 最后一个分区区域不能和坏块替换块区域存在重叠的部分，避免数据冲突。

5 私有数据保存需要划分单独分区

蓝牙、WIFI、用户数据等需要保存时，必须划分单独的分区，并且分区必须以块为单位，不能共用其他分区的存储空间。因为 spi nand flash 擦除是以块为单位的，如果私有数据共用其他分区的存储空间（数据共用了同一个块），那么擦写私有数据时会把原分区数据擦掉，此时如果异常断电，擦掉的数据就不会恢复了。

黑屏案例： STEPLDRA 分区范围是 0x20000~0x40000，大小 128K（占用 1 个块），但是蓝牙配置文件保存在了 0x36000，共用了 STEPLDRA 分区的同一个块。当蓝牙更新配置文件时，如果此时系统异常断电，就会导致分区数据擦除后无法成功回写，STEPLDRA 分区镜像损坏。如果 STEPLDRA 分区是当前的启动分区，那么当系统再次启动时，就会出现 STEPLDRA 分区镜像加载失败，开机黑屏。

6 版本更新说明

2024-06-14 更新：

- 1、优化坏块管理中存在的问题。
- 2、坏块表数据添加 CRC 校验，保证坏块表数据的正确性。
- 3、amt630hv100-freertos 工程中添加擦写缓存机制，更好的对接 FAT 文件系统，提高数据擦写的效率。